

Efficiency

applied on the task of word segmentation

(CC-BY Maria Skeppstedt)

For languages that use white space, the white spaces are typically used for segmenting a text into words (*although this is a simplification. What is a word, really? But that is not the topic for today.*)

The train journey took eight hours

Tågresa tog åtta timmar

However, not all languages use white space.

E.g. Chinese, Japanese

留学生们品尝了使馆厨师做的中国传统佳肴

私の大好きな野菜は牛蒡です

Also, when digitalizing texts through OCR, parts of the text is sometimes incorrectly digitalized, and have lost the white spaces.

hellothere

Other methods for word segmentation are then needed than white space-based.

We will look at how to efficiently implement one method for word segmentation:

1. **Unigram language model:** Only based on the frequency of each individual word

(Later: we will discuss limitation of the unigram language model, and introduce:

2. **N-gram language model:** Based on the frequency of a word in the context of other words.)

Which is the most probable segmentation?

hellothere $\longrightarrow$ hello there

hellothere $\longrightarrow$ hell other e

hellothere $\longrightarrow$ ~~he ll o the re~~

Why?

(You are only allowed to use information on word frequency)

We have a **lexicon** with word frequency (n_w) from a segmented corpus (here, a hypothetical, very small one):

w:	n_w
"a":	120,
"an":	89,
"ape":	2

Also: Add all unknown single letters to the lexicon, with $\underline{n_w} = 1$.

w:	n_w
"a":	120,
"an":	89,
"ape":	2

Also: Add all unknown single letters to the lexicon, with $\underline{n_w} = 1$.

Total number of words in corpus:

$$N = 120 + 89 + 2 = 211$$

Probability of a word:

$$n_w/N \text{ (e.g. "a": } 120/211, \text{ "ape": } 2/211)$$

Probability of a set of words :

$$n_{w1}/N * n_{w2}/N * n_{w3}/N \dots$$

anape:

$$\text{an ape: } 89/211 * 2/211 = 0.0039981$$

$$\text{a n ape: } 120/211 * 1/211 * 2/211 = 0.0000255$$

Winner

First solution: Standard recursion

Our problem has an **optimal substructure**:

(What is an optimal solution to a sub-problem, is also the optimal solution to the overall problem). We use that to write a recursive function.

You don't need to look at "friendofmine" to know how to optimally divide "hellothere"

hellotherefriendofmine

hellotherefriendofmine

→ hello there

You don't need to look at "refriendofmine" to know how to optimally divide "hellothe"

hellotherefriendofmine

→ hello the

We **don't** know which solutions to the subproblems will be used in the **final solution**.

But the best solution to the sub-problems of dividing "hellothere" and "hellothe" is not affected by the part of the string that comes after "hellothere" and "hellothe".

Write the recursive function:

```
def get_best_segmentation_with_score(text):
```

Exemplified on the text: "anape"

```
get_best_segmentation_with_score("anape")
```

```
get_best_segmentation_with_score("anape")
```

Start from the back. The maximum word length in lexicon is 3, so the last word cannot be longer than 3 characters.

anape - Score for "ape" as the last word is:

```
get_best_segmentation_with_score("an")*probability("ape")
```

anape - Score for "pe" as the last word is:

```
get_best_segmentation_with_score("ana")*probability("pe")
```

anape - Score for "e" as the last word is:

```
get_best_segmentation_with_score("anap")*probability("e")
```


an ape - Score for "ape" as the last word is:

w:	n _w
"a":	120,
"an":	89,
"ape":	2

Total: 211

get_best_segmentation_with_score("an")*probability("ape")

0.4218

*

2/211 = 0.003998

("an")

("ape")

Winner!!!!

~~ana~~ pe - ~~Score for "pe" as the last word is:~~

~~get_best_segmentation_with_score("ana")*probability("pe")~~

("pe") not in lexicon, so omit

anap e - Score for "e" as the last word is:

get_best_segmentation_with_score("anap")*probability("e")

0.001137

*

1/211 = 0.000005389

("an a p")

("e")

Base case

A recursive function always needs a base case. When running our function, the string given as an argument gets smaller and smaller each time. Eventually, we will call it with:

```
get_best_segmentation_with_score("")
```

This is our base case, and we need to specify what to do here. An empty string is always segmented into an empty string, i.e., with the probability 1.

```
def get_best_segmentation_with_score(text):  
    if len(text) == 0:  
        return (1, "")
```

First solution: Standard recursion (but with logarithms)

an ape - Score for "ape" as the last word is:

`get_best_segmentation_with_score("an")*probability("ape")`

0.4218

("an")

*

2/211 = 0.003998

("ape")

w: n_w

"a": 120,

"an": 89,

"ape": 2

Total: 211

Problem: The numbers representing the total probabilities get very small for long sequences. These small numbers cannot be represented with normal data types.

anap e - Score for "e" as the last word is:

`get_best_segmentation_with_score("anap")*probability("e")`

0.001137

("an a p")

*

1/211 = 0.000005389

("e")

One solution: Use the logarithm of every number you deal with.

One solution: Instead use the **logarithm** of every number you deal with.

w:	n_w	$\log(n_w)$
"a":	120	4.79
"an":	89	4.49
"ape":	2	0.70

Total number of words in corpus:

$$N = 120 + 89 + 2 = 211$$

$$\log(N) = 5.35$$

Then: / turns into –
and * turns into +

Logarithmic probability of a word: $\log(n_w) - \log(N)$ (Instead of n_w/N)

Logarithmic probability of a set of words :

$$(\log(n_{w1}) - \log(N)) + (\log(n_{w2}) - \log(N)) + (\log(n_{w3}) - \log(N)) + \dots$$

$$(Instead\ of: n_{w1}/N * n_{w2}/N * n_{w3}/N \dots)$$

an ape - Score for "ape" as the last word is:

w:	n_w	$\log(n_w)$
"a":	120	4.79
"an":	89	4.49
"ape":	2	0.70

Total: $\log(N) = 5.35$

$\text{get_best_segmentation_with_score}(\text{"an"}) * \text{probability}(\text{"ape"})$

-0.863

("an")

+

$(0.7 - 5.35) = -5.513$

("ape")

Winner!!!!

~~ana~~ pe - ~~Score for "pe" as the last word is:~~

~~$\text{get_best_segmentation_with_score}(\text{"ana"}) * \text{probability}(\text{"pe"})$~~

("pe") not in lexicon, so omit

anap e - Score for "e" as the last word is:

$\text{get_best_segmentation_with_score}(\text{"anap"}) * \text{probability}(\text{"e"})$

$(\log(1) = 0)$

-6.779

("an a p")

+

$(\log(1) - 5.35) = -12.129$

("e")

Update the base case

A recursive function always needs a base case. When running our function, the string given as an argument gets smaller and smaller each time. Eventually, we will call it with:

```
get_best_segmentation_with_score("")
```

This is our base case, and we need to specify what to do here. An empty string is always segmented into an empty string, i.e., with the probability 1. $(\log(1) = 0)$

```
def get_best_segmentation_with_score(text):  
    if len(text) == 0:  
        return (log(1), "")
```

Second solution: Recursion with dynamic programming

Problem: Our solution is very slow for longer strings.
Solve this, by avoiding to compute the same solution many times, i.e. use: **dynamic programming**

Appropriate, because:

- A. Optimal substructure:
(What is an optimal solution to a sub-problem, is also the optimal solution to the overall problem)
- B. Overlapping sub-problems:
(The same sub-problems need to be solved over-and-over again. So the solution to these sub-problems can be computed once, and stored)

Use the same recursive algorithm, except:

- Whenever, a segmentation is computed:
Store this segmentation, and its probability, in a table.
- Before computing how to best segment a text:
Check if a solution for this text can be retrieved from your table.

Text to segment	Best segmentation	Log probability
""	""	0
"a"	"a"	-0.56
"an"	"an"	-0.86
"ana"	"an a"	-1.43
"anap"	"an a p"	-6.78
"anape"	"an ape"	-5.52

Third solution: Iterative solution, with dynamic programming

If the recursive depth gets very deep, a recursive solution will require very much memory. It can, therefore, sometimes be better with an **iterative** solution.

For this case, an iterative solution can be constructed by starting from the beginning of the text (""), and iteratively construct the table below.

Text to segment	Best segmentation	Log probability
""	""	0
"a"	"a"	-0.56
"an"	"an"	-0.86
"ana"	"an a"	-1.43
"anap"	"an a p"	-6.78
"anape"	"an ape"	-5.52

Iteratively fill the table:

get_best_segmentation_with_score_iterative("", result_table)

Text to segment	Best segmentation	Log probability
""	""	0

get_best_segmentation_with_score_iterative("a", result_table)

Text to segment	Best segmentation	Log probability
""	""	0
"a"	"a"	-0.56

get_best_segmentation_with_score_iterative("an", result_table)

Text to segment	Best segmentation	Log probability
""	""	0
"a"	"a"	-0.56
"an"	"an"	-0.86

get_best_segmentation_with_score_iterative("ana", result_table)

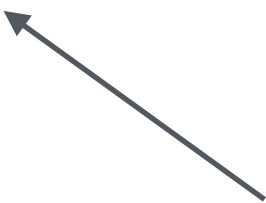
Text to segment	Best segmentation	Log probability
""	""	0
"a"	"a"	-0.56
"an"	"an"	-0.86
"ana"	"an a"	-1.43

get_best_segmentation_with_score_iterative("anap", result_table)

Text to segment	Best segmentation	Log probability
""	""	0
"a"	"a"	-0.56
"an"	"an"	-0.86
"ana"	"an a"	-1.43
"anap"	"an a p"	-6.78

get_best_segmentation_with_score_iterative("anape", result_table)

Text to segment	Best segmentation	Log probability
""	""	0
"a"	"a"	-0.56
"an"	"an"	-0.86
"ana"	"an a"	-1.43
"anap"	"an a p"	-6.78
"anape"	"an ape"	-5.52

 **Winner!!!!**

```
get_best_segmentation_with_score_iterative("anape", result_table)
```

Can be implemented very similar to the recursive version.

But instead of making a recursive call:

We do a look-up the best segmentation for string before the potential word in our table.



Old recursive:

an ape - Score for "ape" as the last word is:

$\text{get_best_segmentation_with_score}(\text{"an"}) * \text{probability}(\text{"ape"})$

w:	n_w	$\log(n_w)$
"a":	120	4.79
"an":	89	4.49
"ape":	2	0.70

Total: $\log(N) = 5.35$

$$\underbrace{-0.863}_{(\text{"an"})} + \underbrace{(0.7 - 5.35)}_{(\text{"ape"})} = -5.513$$

Winner!!!!

~~ana~~ pe - ~~Score for "pe" as the last word is:~~

~~$\text{get_best_segmentation_with_score}(\text{"ana"}) * \text{probability}(\text{"pe"})$~~
~~*("pe") not in lexicon, so omit*~~

anap e - Score for "e" as the last word is:

$\text{get_best_segmentation_with_score}(\text{"anap"}) * \text{probability}(\text{"e"})$ $(\log(1) = 0)$

$$\underbrace{-6.779}_{(\text{"an a p"})} + \underbrace{(\log(1) - 5.35)}_{(\text{"e"})} = -12.129$$

an ape - Score for "ape" as the last word is:

look_up_in_table("an") * probability("ape")

$$\underbrace{-0.863}_{("an")} + \underbrace{(0.7 - 5.35)}_{("ape")} = -5.513$$

w:	n _w	log(n _w)
"a":	120	4.79
"an":	89	4.49
"ape":	2	0.70

Total: log(N) = 5.35

Winner!!!!

~~ana pe - Score for "pe" as the last word is:~~

~~get_best_segmentation_with_score("ana") * probability("pe")~~
 ("pe") not in lexicon, so omit

anap e - Score for "e" as the last word is:

look_up_in_table("anap") * probability("e")

(log(1) = 0)

$$\underbrace{-6.779}_{("an a p")} + \underbrace{(\log(1) - 5.35)}_{("e")} = -12.129$$

Old recursive:

an ape - Score for "ape" as the last word is:

`get_best_segmentation_with_score("an")*probability("ape")`

w:	n _w	log(n _w)
"a":	120	4.79
"an":	89	4.49
"ape":	2	0.70

Total: log(N) = 5.35

$$\underbrace{-0.863}_{("an")} + \underbrace{(0.7-5.35)}_{("ape")} = -5.513$$

Winner!!!!

New iterative:

an ape - Score for "ape" as the last word is:

`look_up_in_table("an")*probability("ape")`

$$\underbrace{-0.863}_{("an")} + \underbrace{(0.7-5.35)}_{("ape")} = -5.513$$

Winner!!!!